

Gradient Boosting on Trees for Time Series Forecasting: Best Practices

Wojciech Widel *

08.03.2026

1 Data Strategy

- **Stationarity (The Trend Trap):** Tree models cannot extrapolate trends. Always difference the target ($y_t - y_{t-1}$) or detrend data before training. This forces the model to predict *changes* rather than absolute values.
- **Global Modeling:** Do not train one model per time series. Pool all products/locations into a single dataset. Use IDs (e.g., `store_id`) as categorical features to share statistical strength across the portfolio.
- **Avoid Leakage:** When creating lag features, simulate the “data availability” cutoff perfectly. A prediction for week t can only use data available strictly before t .
- **Transformation Order:** Sales data is typically multiplicative (variance grows with volume).
 1. **Log Transform:** Apply `log1p(target)` to stabilize variance (converts multiplicative to additive).
 2. **Difference:** Apply differencing ($y_t - y_{t-1}$) to remove trend (stabilizes the mean).

Invert the process for final predictions: Cumulative sum $\rightarrow$ Exponential.

2 Forecasting Strategy

- **Recursive (Iterative):** Train a single model for $t + 1$. Feed predictions back as inputs for subsequent steps ($t + 2, \dots, t + h$).
 - *Pros:* Maximum data utilization; single model to maintain; flexible horizon.
 - *Cons:* Error accumulation (compounding errors); distribution shift (training on actuals, predicting on predictions).
- **Direct:** Train h separate models. Model i predicts step $t + i$ directly using actual history available at t .
 - *Pros:* No error accumulation; consistent train/test distributions; optimized for specific horizons.
 - *Cons:* High computational cost; ignores temporal dependencies between steps (can result in jagged forecasts).
- **Hybrid (DirRec):** Train h models, but pass predictions from model $i - 1$ as a feature to model i .
 - *Pros:* Captures sequential dependencies while mitigating error propagation.
 - *Cons:* Complex pipeline; requires careful feature engineering for each step.

*With the help of `z.ai`.

3 Feature Engineering

- **Seasonality: Lags vs. Fourier:**
 - *Lag Features* (e.g., `lag_52`): Excellent for high-autocorrelation but drops initial observations (data loss). Trees cannot extrapolate values beyond the training range.
 - *Fourier Terms*: Sine/Cosine pairs capture smooth periodic patterns without data loss. Allows trend extrapolation independent of seasonality shape. Best for short histories.
- **Cyclic Encoding:** Do not use raw integers for time (e.g., 1-12). Use Sine/Cosine transformations ($\sin(2\pi \frac{\text{month}}{12})$) so that cyclic time points (Dec $\rightarrow$ Jan) are mathematically close.
- **Holiday Windows:** Binary flags are insufficient. Create features like `days_to_holiday` and `days_since_holiday` to capture sales ramps.
- **Rolling Statistics:** Generate `rolling_mean`, `rolling_std`, and `rolling_max` to provide the model with recent context and volatility measures.
- **The Exogenous Trap:** Only include features (e.g., Price, Weather) if their future values are known or can be reliably forecasted. Using “actual future weather” creates unreplicable leakage in production.

4 Evaluation Metrics

- **BIAS (Mean Error):**

$$\text{BIAS} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Measures systematic over/under-forecasting.

- *Sign*: Positive value = Over-forecast (predicting higher than actual).
- *Scaling*: $\text{BIAS}\% = \text{BIAS} / \bar{y}_{\text{train}}$. Allows comparison across series of different scales.
- *Context*: Critical for inventory. Positive bias $\rightarrow$ overstocking; Negative bias $\rightarrow$ stockouts.

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Average magnitude of errors. Robust to outliers.

- *Theory*: Minimizing MAE estimates the **median**.
- *Scaling*: $\text{MAE}\% = \text{MAE} / \bar{y}_{\text{train}}$.
- *Context*: Preferred for stable demand where the cost of an error is linear (e.g., one missed sale costs \$X).

- **RMSE (Root Mean Squared Error):**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Penalizes large errors more heavily.

- *Theory*: Minimizing RMSE estimates the **mean**. Sensitive to outliers (spikes).
- *Context*: Preferred when large errors are disproportionately costly (e.g., catastrophic stockouts) or if you prefer the model to predict the average demand.

- **MASE (Mean Absolute Scaled Error):**

$$\text{MASE} = \frac{\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|}{\frac{1}{N-1} \sum_{t=2}^N |y_t - y_{t-1}|}$$

Scales error by the in-sample naive forecast (one-step drift). The denominator is calculated on the training set.

- *Interpretation*: < 1.0 means better than naive benchmark; > 1.0 means worse.
- *Context*: Best for intermittent demand or comparing series with vastly different scales/levels. Handles zeros gracefully (unlike MAPE).

5 Model Configuration

- **Categorical Features:** Avoid One-Hot Encoding for high-cardinality IDs (e.g., Store ID). Use LightGBM's native categorical handling (`dtype="category"`) or regularized Target Encoding.
- **Objective Function:**
 - *Point Forecast:* Use `regression_l1` (MAE) for robustness to outliers, or `tweedie` for sales data with many exact zeros.
 - *Uncertainty:* Use `quantile` objective with $\alpha = 0.05$ (pessimistic) and $\alpha = 0.95$ (optimistic) to generate prediction intervals for inventory planning.
- **Regularization:** Keep `min_child_samples` (> 20) and `feature_fraction` (< 1.0) reasonable to prevent memorization of specific time steps.

6 Training & Tuning

- **The "Lower Learning Rate" Trick:** After hyperparameter search, manually lower the `learning_rate` (e.g., by 5x) and increase `num_boost_round` accordingly. Use early stopping to find the optimal iteration count for this finer learning rate.
- **Dataset Partitioning:** Set aside the last h periods as the Test Set (Final Exam). Do not use random K-Fold.
- **Cross-Validation:** Use `TimeSeriesSplit` or Rolling Origin Cross-Validation. Move the training cutoff forward in steps to validate on multiple time windows.
- **The "Gap":** If there is a data latency in production (e.g., data available 1 week late), introduce a gap between the training end and validation start to simulate this reality.